Unit 1: 8086 Architecture

- Evolution of Microprocessors and types, 8086 Microprocessor, Salient features, Pin descriptions, Architecture of 8086 - Functional Block diagram , Register organization, Concepts of pipelining(5 stage), Memory segmentation, Physical memory addresses generation.,
  - Minimum mode and Maximum Mode of 8086

8086 Microprocessor Features

- It is a 16 bit µp.(Width of data bus=16 bit)
  • 8086 has a 20 bit address bus can access up to memory locations (1 MB) .
  • It can support upto 64K I/O ports.
  • It provides 14, 16-bit registers.
  • It has multiplexed address and data bus AD0- AD15 and A16 – A19
- Execute stage executes these instructions.

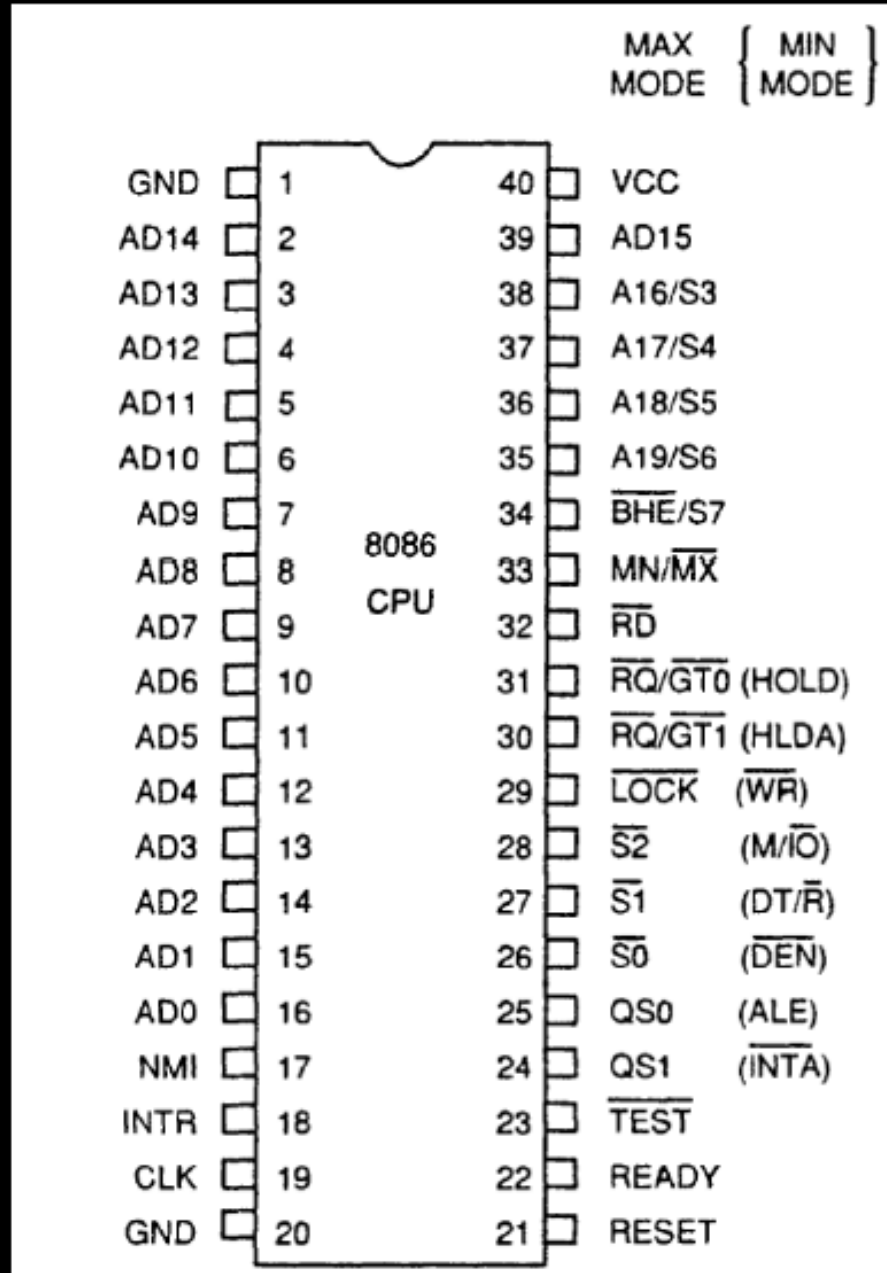
8086 Microprocessor

- It requires single phase clock with 33% duty cycle(TON/TON+TOFF) to provide internal timing.
- 8086 is designed to operate in two modes, **Minimum and Maximum.**
- **It uses two stages of pipelining, i.e. Fetch Stage and Execute Stage,** which improves performance. Fetch stage can prefetch up to 6 bytes of instructions from memory and queues them in order to speed up instruction execution.
- It requires +5V power supply.
- A 40 pin dual in line package.

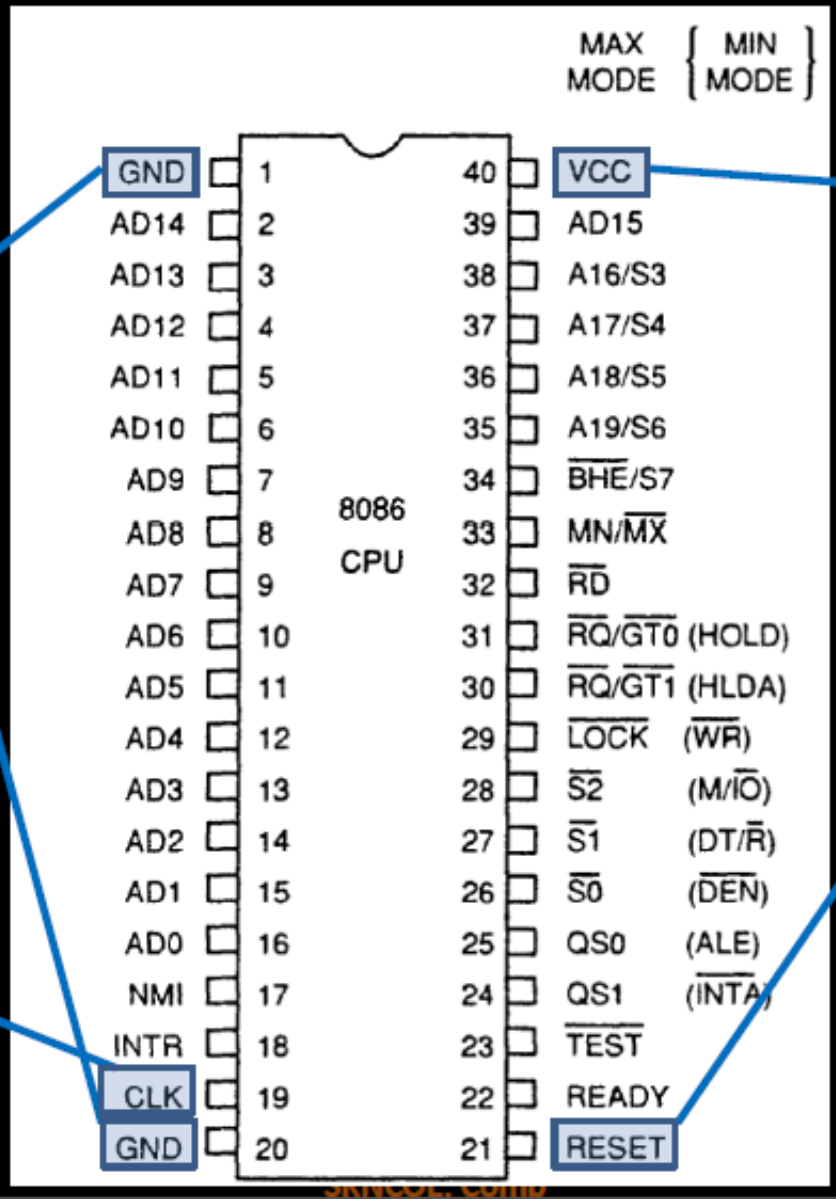
Minimum and Maximum Modes:

- The minimum mode is selected by applying logic 1 to the MN / ~MX input pin. This is a single microprocessor configuration.
- The maximum mode is selected by applying logic 0 to the MN / ~MX input pin. This is a multi micro processors configuration.

INTEL 8086 - Pin Diagram



INTEL 8086 - Pin Details



Ground

Power Supply

5V ± 10%

Clock

Duty cycle: 33%

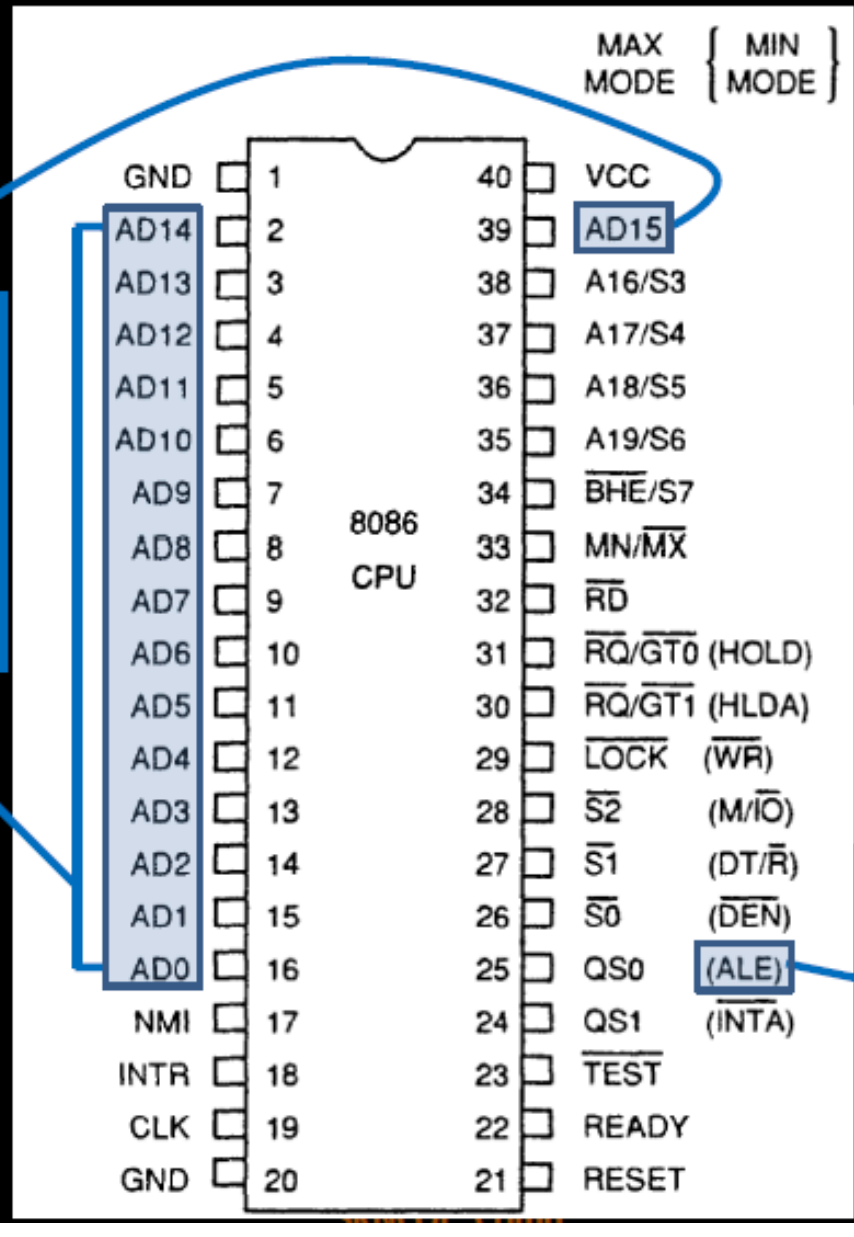
Reset

Registers, seg regs, flags

CS: FFFFH, IP: 0000H

If high for minimum 4 clks

INTEL 8086 - Pin Details



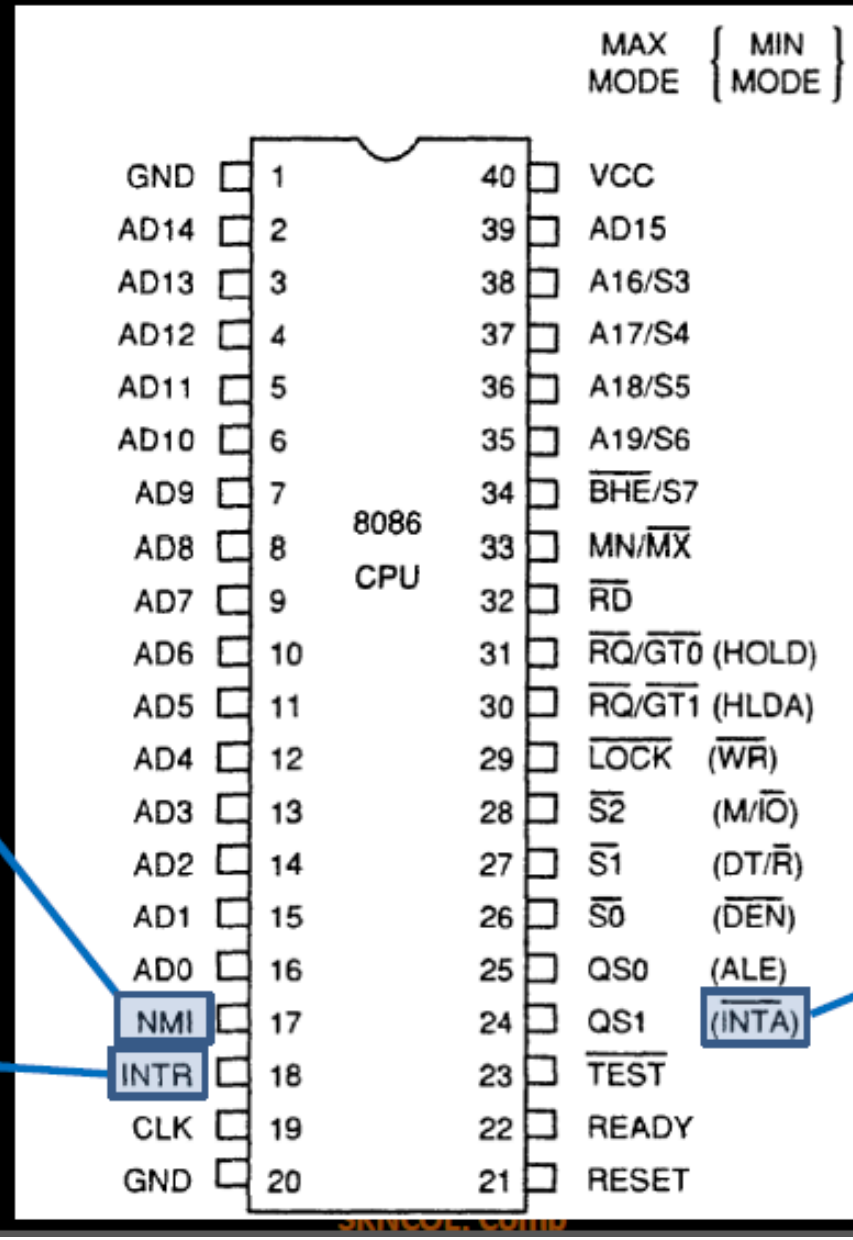
Address/Data Bus:

Contains address bits A_{15}-A_0 when ALE is 1 & data bits D_{15} – D_0 when ALE is 0.

Address Latch Enable:

When high, multiplexed address/data bus contains address information.

INTEL 8086 - Pin Details



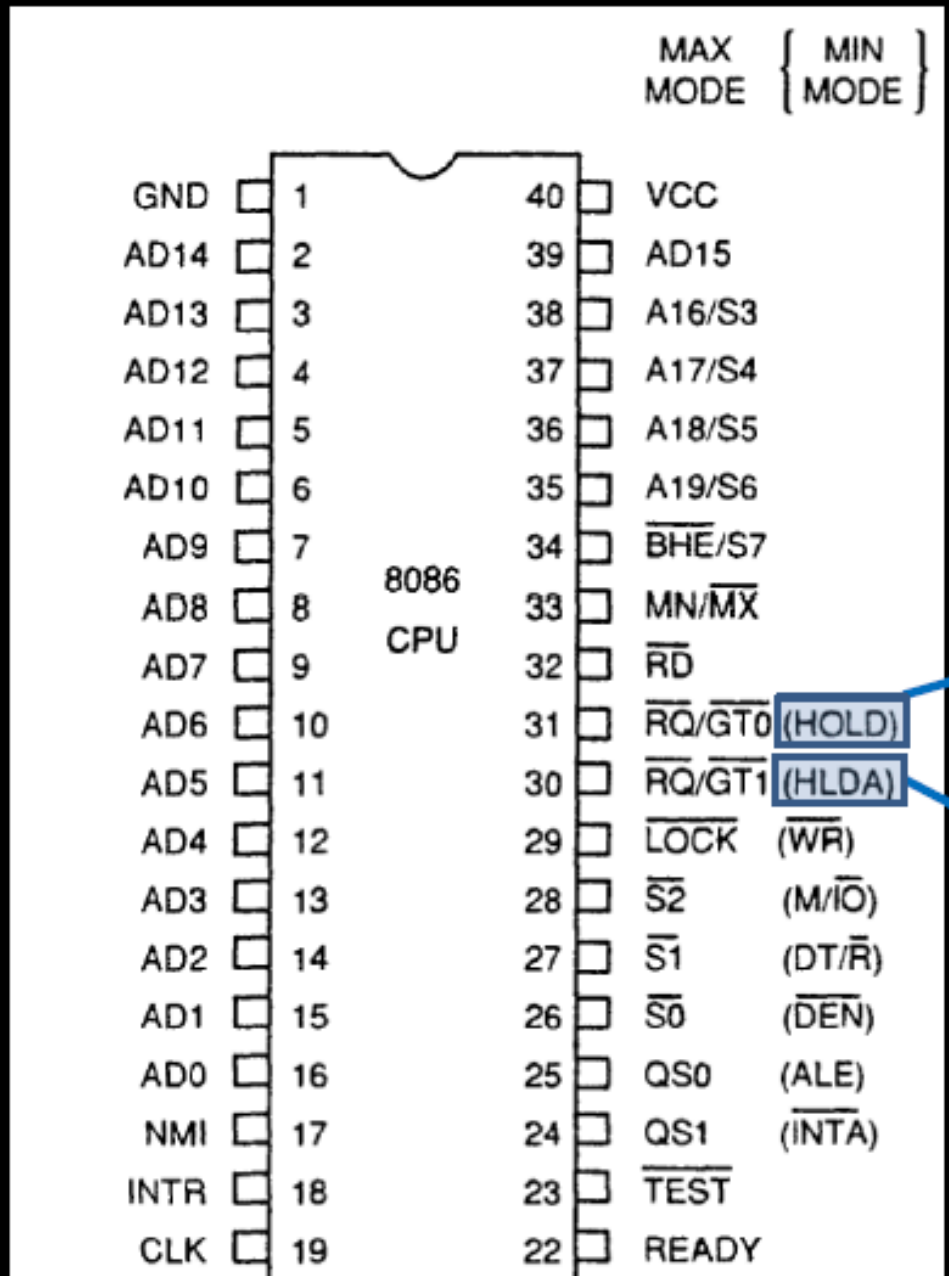
INTERRUPT

Non - maskable interrupt

Interrupt request

Interrupt acknowledge

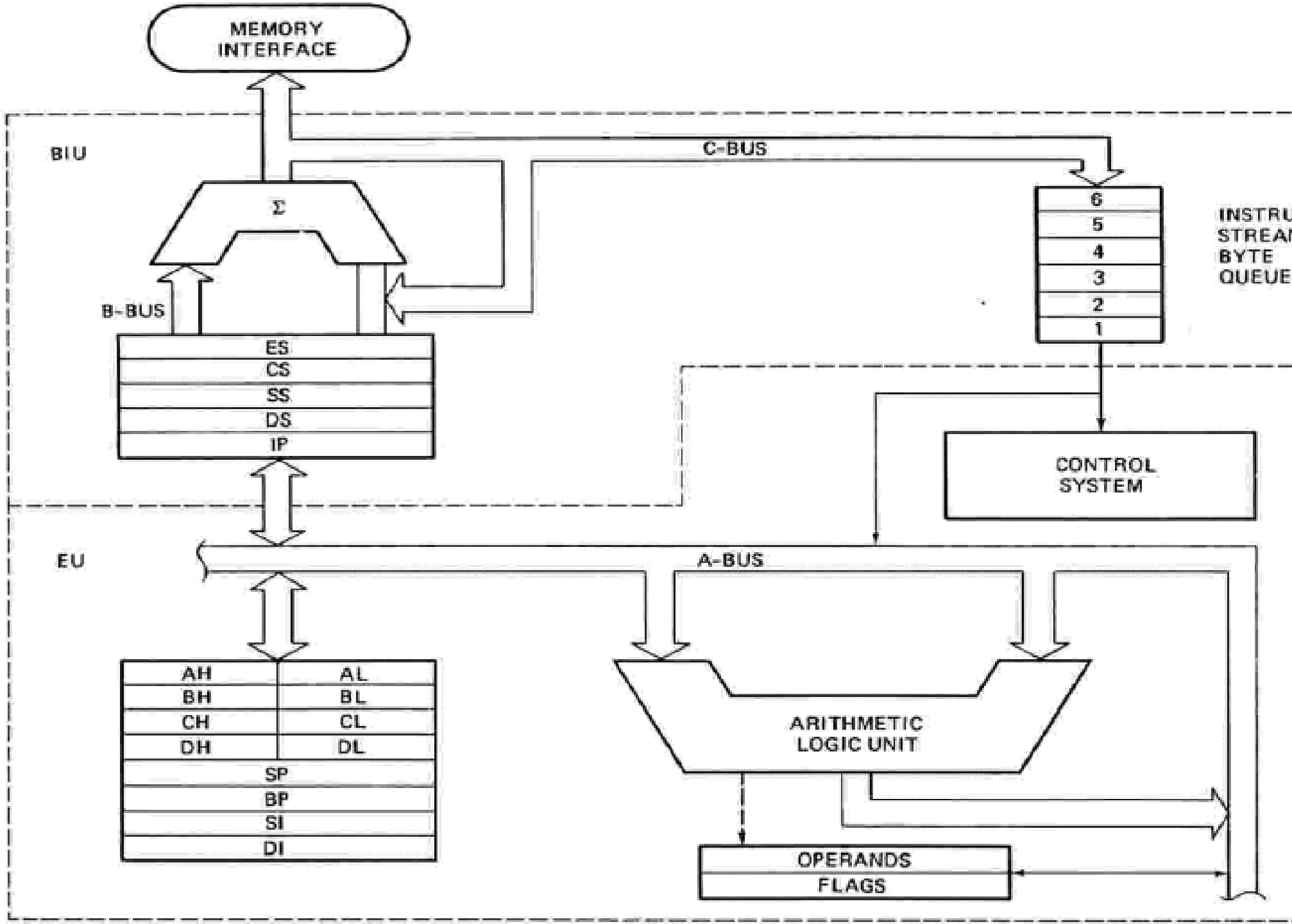
INTEL 8086 - Pin Details



Direct Memory Access

Hold

Hold acknowledge



URE 2-7 8086 internal block diagram. (*Intel Corp.*)

Internal Architecture of 8086

- 8086 has two blocks BIU and EU.
- The BIU (Bus Interface unit)performs all bus operations such as instruction fetching, reading and writing operands for memory and calculating the addresses of the memory operands.
- The instruction bytes are transferred to the instruction queue.
- EU executes instructions from the instruction system byte queue.

Internal Architecture of 8086

- Both units operate asynchronously to give the 8086 an overlapping instruction fetch and execution mechanism which is called as Pipelining. This results in efficient use of the system bus and system performance.
  - BIU contains Instruction queue, Segment registers, Instruction pointer, Address adder.
  - EU contains Control circuitry, Instruction decoder, ALU, Pointer and Index register, Flag register.

Internal Architecture of 8086

Bus Interface Unit:

- It provides a full 16 bit bidirectional data bus and 20 bit address bus.
- The bus interface unit is responsible for performing all external bus operations.

Specifically it has the following functions:

- Instruction fetch, Instruction queuing, Operand fetch and storage, Address relocation and Bus control.
- The BIU uses a mechanism known as an instruction stream queue to implement a **pipeline architecture**.

Internal Architecture of 8086

- **This queue permits prefetch of up to six bytes of instruction code**. When ever the queue of the BIU is not full, it has room for at least two more bytes and at the same time the EU is not requesting it to read or write operands from memory, the BIU is free to look ahead in the program by prefetching the next sequential instruction.
- These prefetching instructions are held in its FIFO queue. With its 16 bit data bus, the BIU fetches two instruction bytes in a single memory cycle.
- After a byte is loaded at the input end of the queue, it automatically shifts up through the **FIFO** to the empty location nearest the output.

Internal Architecture of 8086

- The EU accesses the queue from the output end. It reads one instruction byte after the other from the output of the queue. If the queue is full and the EU is not requesting access to operand in memory.
- These intervals of no bus activity, which may occur between bus cycles are known as ***Idle state***.
- If the BIU is already in the process of fetching an instruction when the EU request it to read or write operands from memory or I/O, the BIU first completes the instruction fetch bus cycle before initiating the operand read / write cycle.

Internal Architecture of 8086

- **The BIU also contains a dedicated adder which is used to generate the 20 bit physical address that is output on the address bus.**
- For example, the physical address of the next instruction to be fetched is formed by combining the current contents of the code segment CS register and the current contents of the instruction pointer IP register.
- The BIU is also responsible for generating bus control signals such as those for memory read or write and I/O read or write.

Internal Architecture of 8086

• **EXECUTION UNIT** : The Execution unit is responsible for **decoding and executing all instructions.**

• The EU extracts instructions from the top of the queue in the BIU, decodes them, generates operands if necessary, passes them to the BIU and requests it to perform the read or write byte cycles to memory or I/O and perform the operation specified by the instruction on the operands.

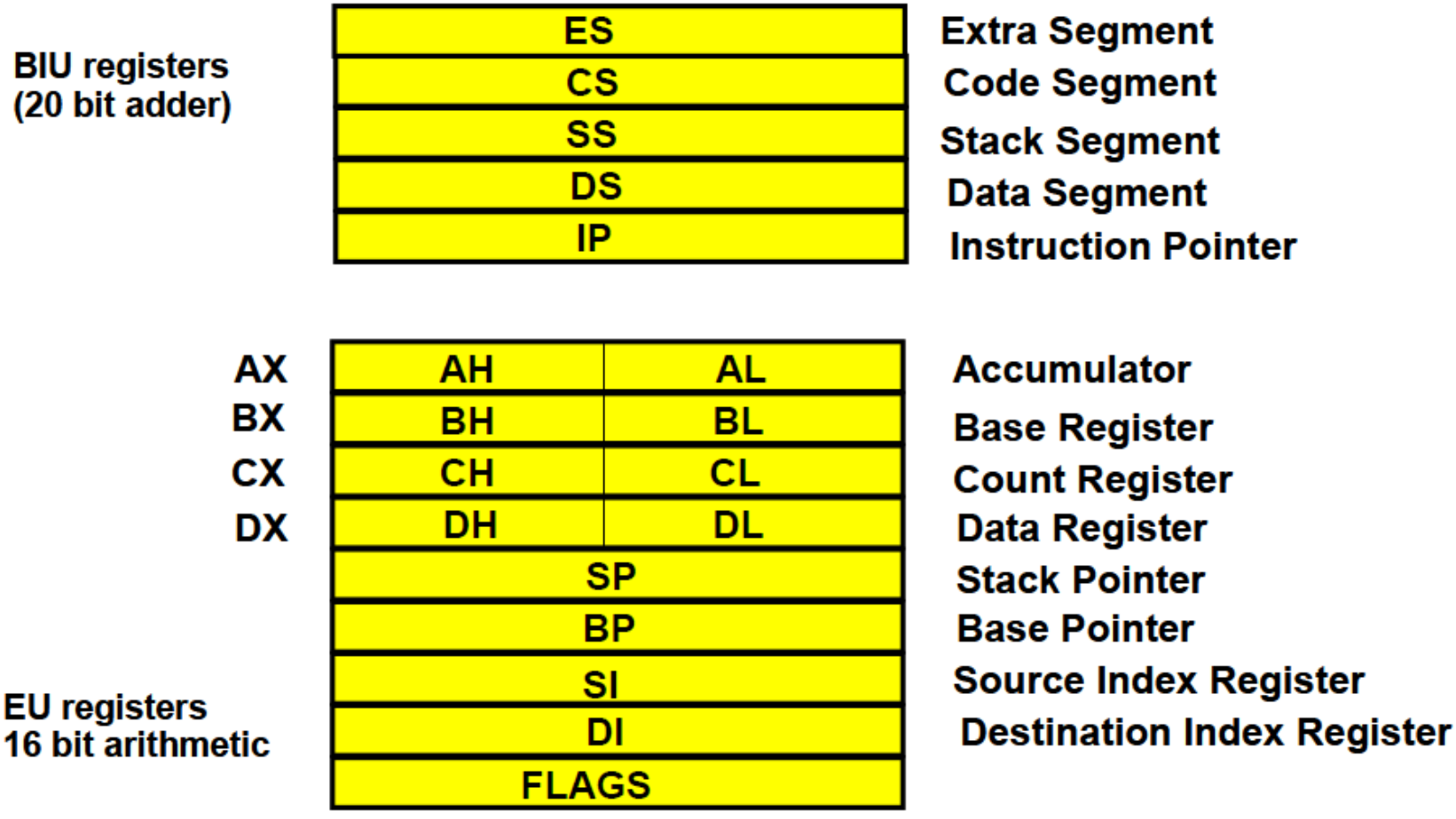
• During the execution of the instruction, the **EU tests the status and control flags and updates them based on the results of executing the instruction.**

Internal Architecture of 8086

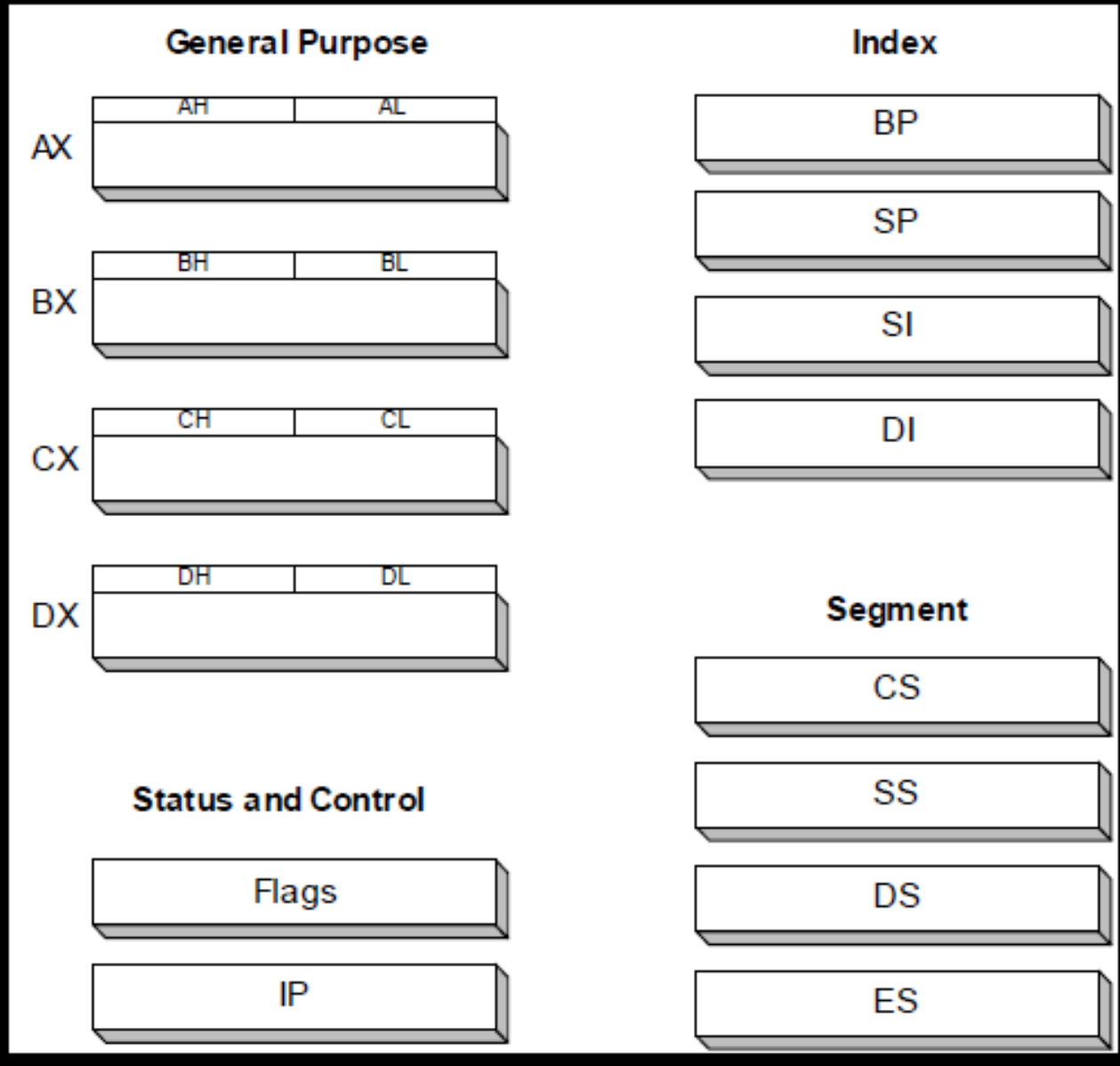
- If the queue is empty, the EU waits for the next instruction byte to be fetched and shifted to top of the queue.
- **When the EU executes a branch or jump instruction, it transfers control to a location corresponding to another set of sequential instructions.**
- Whenever this happens, the BIU automatically resets the queue and then begins to fetch instructions from this new location to refill the queue.

Register organization of 8086

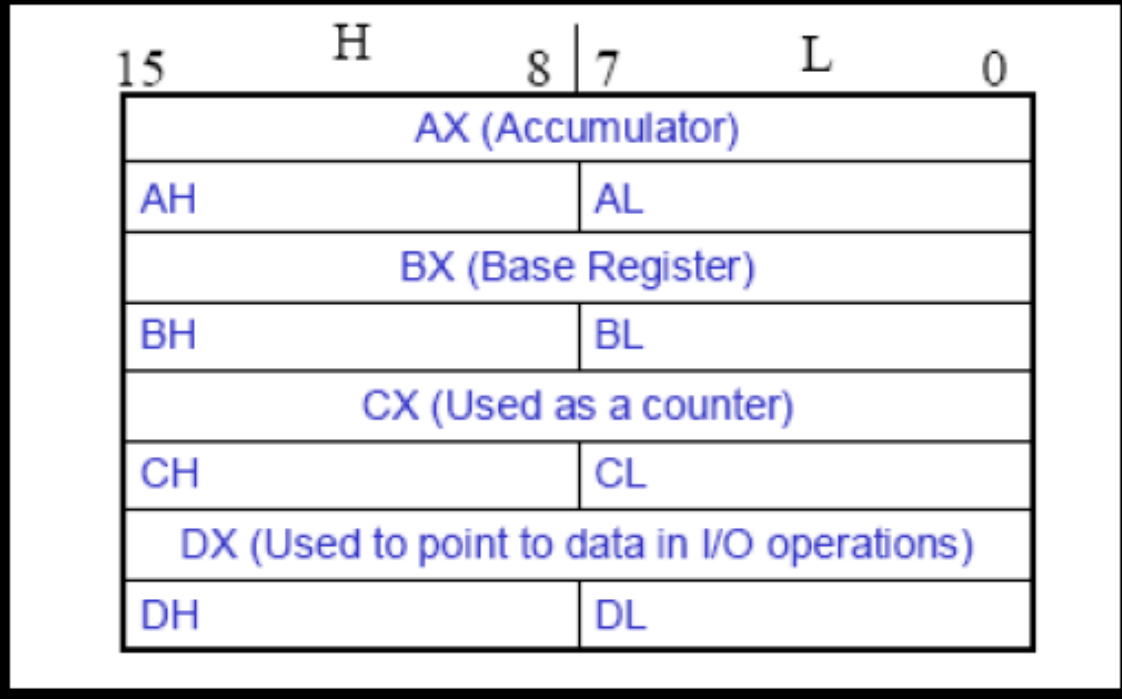
- The 8086 has four groups of the user accessible internal registers. They are the **instruction pointer, four data registers, four pointer and index register, four segment registers.**
- The 8086 has a total of fourteen 16-bit registers including a 16 bit register called the ***status register***, with 9 of bits implemented for status and control flags.


BIU registers
(20 bit adder)
ES
CS
SS
DS
IP
Extra Segment
Code Segment
Stack Segment
Data Segment
Instruction Pointer
AX
BX
CX
DX
AH
AL
BH
BL
CH
CL
DH
DL
SP
BP
SI
DI
FLAGS
Accumulator
Base Register
Count Register
Data Register
Stack Pointer
Base Pointer
Source Index Register
Destination Index Register
EU registers
16 bit arithmetic

8086 Registers



General Purpose Registers



AX - the Accumulator
BX - the Base Register
CX - the Count Register
DX - the Data Register

- Normally used for storing temporary results
- Each of the registers is 16 bits wide **(AX, BX, CX, DX)**
- Can be accessed as either 16 or 8 bits AX, AH, AL

General Purpose Registers

- **AX**
  - Accumulator Register
  - Preferred register to use in arithmetic, logic and data transfer instructions because it generates the shortes Machine Language Code
  - Must be used in multiplication and division operations
  - Must also be used in I/O operations

- **BX**
  - Base Register
  - Also serves as an address register

General Purpose Registers

- **CX**
  - Count register
  - Used as a loop counter
  - Used in shift and rotate operations
- **DX**
  - Data register
  - Used in multiplication and division
  - Also used in I/O operations

Pointer and Index Registers

SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Destination Index
IP	Instruction Pointer

- All 16 bits wide, L/H bytes are not accessible
- Used as memory pointers
  - Example: MOV AH, [SI]
    - *Move the byte stored in memory location whose address is contain register SI to register AH*

Flag Register

- Is a flip flop – which indicates some condition – produced by execution if instruction or controls certain operations of EU

Flag Register

Overflow

Direction

Interrupt enable

Trap

Sign

Zero

Auxiliary Carry

Parity

Carry

6 are status flags

3 are control flag

- **Flags** is a 16-bit register containing 9 one bit flags.
- ***SF*- Sign Flag:** This flag is set, when the result of any computation is negative. For signed computations the sign flag equals the MSB of the result.
- ***ZF*- Zero Flag:** This flag is set, if the result of the computation or comparison performed by the previous instruction is zero.
- ***PF*- Parity Flag:** This flag is set to 1, if the lower byte of the result contains even number of 1's.
- ***CF*- Carry Flag:** This flag is set, when there is a carry out of MSB in case of addition or a borrow in case of subtraction.
- ***AF*-Auxilary Carry Flag:** This is set, if there is a carry from the lowest nibble, i.e, bit three during addition, or borrow for the lowest nibble, i.e, bit three, during subtraction.

Flag Register Continued…

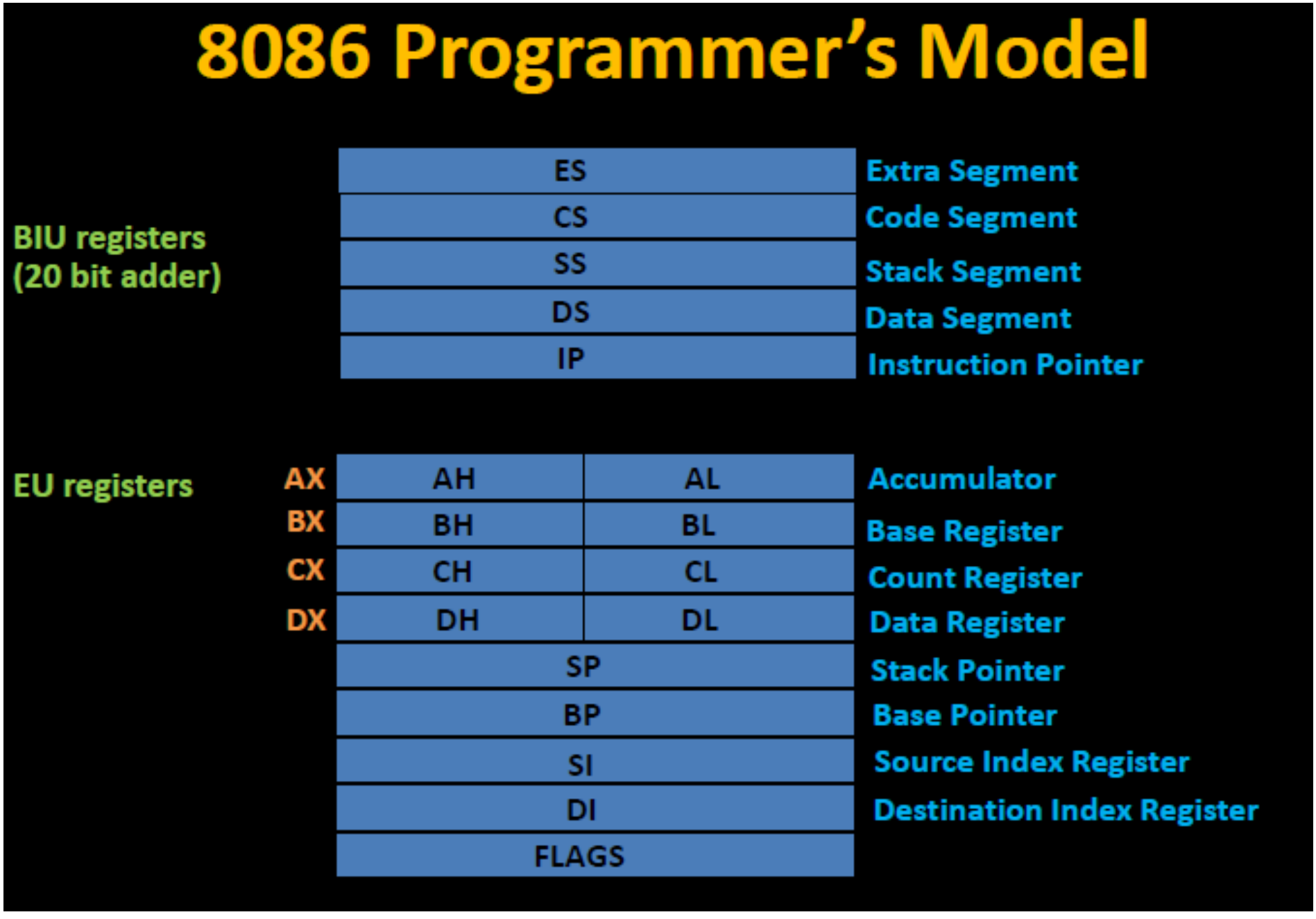
- ***OF*- Over flow Flag:** This flag is set, if an overflow occurs, i.e, if the result of a signed operation is large enough to accommodate in a destination register. The result is of more than 7-bits in size in case of 8-bit signed operation then the overflow will be set.
- ***TF*- Trap Flag:** If this flag is set, the processor enters the **single step execution mode**. The processor executes the current instruction and the control is transferred to the Trap interrupt service routine.
- ***IF*- Interrupt Flag:** If this flag is set, the maskable interrupts are recognized by the CPU, otherwise they are ignored.
- ***D*- Direction Flag:** This is used by string manipulation instructions. If this flag bit is '0', the string is processed beginning from the lowest address to the highest address, i.e., auto incrementing mode. Otherwise, the string is processed from the highest address towards the lowest address, i.e., auto decrementing mode.

Programming model of the 8086

- The programming model of the 8086 is considered to be program visible because its registers are used during application programming and are specified by the instructions. Figure below illustrates the programming model of 8086 microprocessor.

- Some registers are general-purpose or multipurpose registers, while some have special purposes

8086 Programmer's Model



Segment registers

- The four segment registers are CS, DS, ES and SS—standing for code segment register, data segment register, extra segment register and stack segment register respectively.
- When a particular memory is being read or written into, the corresponding memory address is determined by the content of one of these four segment registers in conjunction with their offset addresses.
- The contents of these registers can be changed so that the program may jump from one active code segment to another one.
- The use of these segment registers will be more apparent in memory segmentation schemes.

Segment registers

- Most of the registers contain data/instruction offsets within 64 KB memory segment.
- There are four different 64 KB segments for instructions, stack, data and extra data.
- To specify where in 1 MB of processor memory these 4 segments are located the processor uses four segment registers:

Segment registers

- **Code segment** (CS) is a 16-bit register containing address of 64 KB segment with processor instructions.
- The processor uses CS segment for all accesses to instructions referenced by instruction pointer (IP) register.
- CS register cannot be changed directly. The CS register is automatically updated during far jump, or call and return instructions

Segment registers

- **Stack segment (SS)** is a 16-bit register containing address of 64KB segment with program stack.
- By default, the processor assumes that all data referenced by the stack pointer (SP) and base pointer (BP) registers is located in the stack segment.
- SS register can be changed directly using POP instruction

Segment registers

- **Data segment** (DS) is a 16-bit register containing address of 64KB segment with program data.
- By default, the processor assumes that all data referenced by general registers (AX, BX, CX, DX) and index register (SI, DI) is located in the data segment.
- DS register can be changed directly using POP.

Segmentation

- **Segmentation** is the process in which the main memory of the computer is divided into different segments and each segment has its own base address.
- It is basically used to enhance the speed of execution of the computer system, so that processor is able to fetch and execute the data from the memory easily and fast.

Need for Segmentation –

The Bus Interface Unit (BIU) contains four 16 bit special purpose registers (mentioned below) called as Segment Registers.

- **Code segment register (CS):** is used for addressing memory location in the code segment of the memory, where the executable program is stored.
- **Data segment register (DS):** points to the data segment of the memory where the data is stored.
- **Extra Segment Register (ES):** also refers to a segment in the memory which is another data segment in the memory.
- **Stack Segment Register (SS):** is used for addressing stack segment of the memory. The stack segment is that segment of memory which is used to store stack data.

Need for Segmentation –

- The number of address lines in 8086 is 20, 8086 BIU will send 20bit address, so as to access one of the 1MB memory locations.
- The four segment registers actually contain the upper 16 bits of the starting addresses of the four memory segments of 64 KB each with which the 8086 is working at that instant of time.
- A segment is a logical unit of memory that may be up to 64 kilobytes long. Each segment is made up of contiguous memory locations. It is independent, separately addressable unit. Starting address will always be changing. It will not be fixed.
- **Note that the 8086 does not work the whole 1MB memory at any given time. However it works only with four 64KB segments within the whole 1MB memory.**

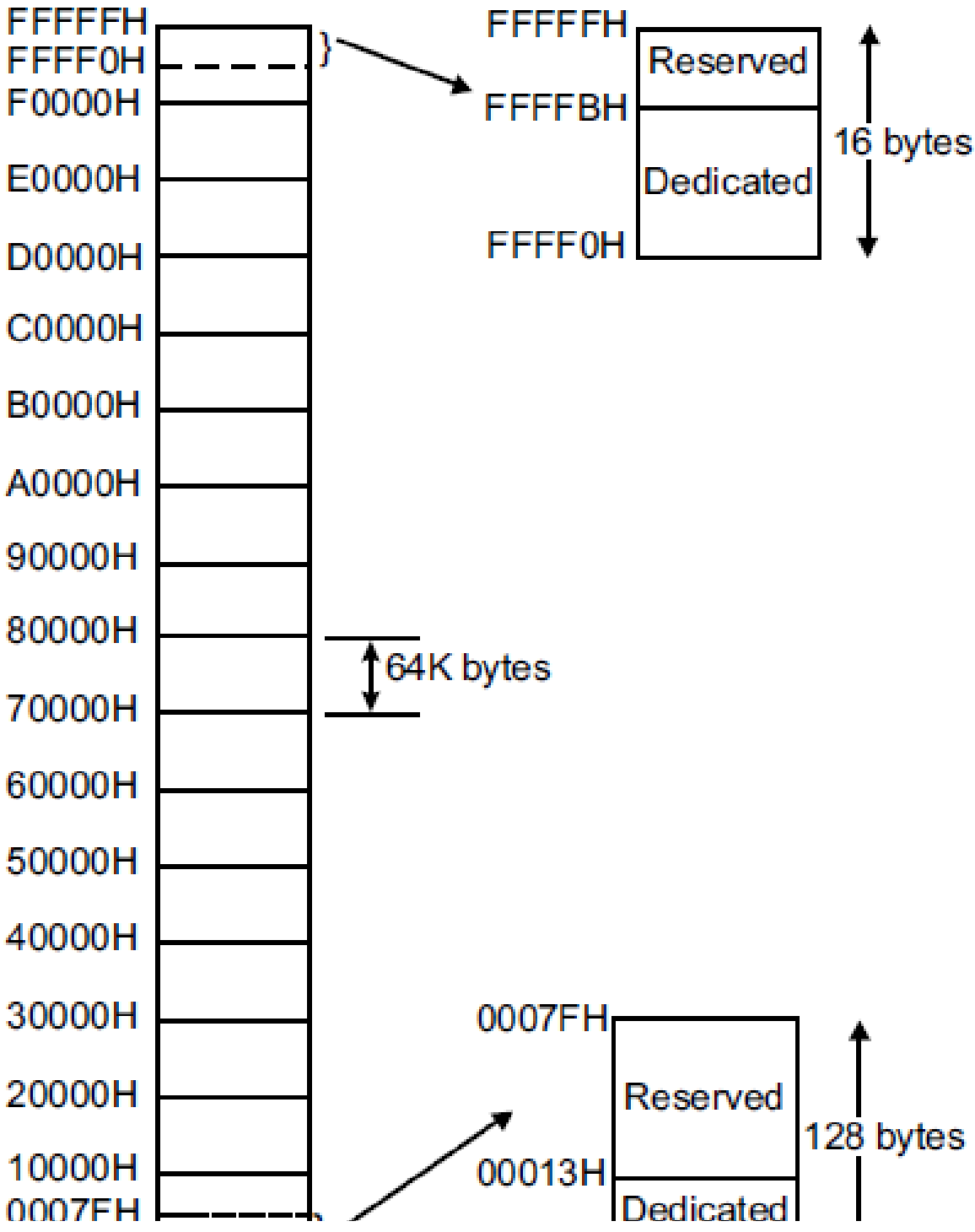
Advantages of the Segmentation

The main advantages of segmentation are as follows:

- It provides a powerful memory management mechanism.
- Data related or stack related operations can be performed in different segments.
- Code related operation can be done in separate code segments.
- It allows to processes to easily share data.
- It allows to extend the address ability of the processor, i.e. segmentation allows the use of 16 bit registers to give an addressing capability of 1 Megabytes. Without segmentation, it would require 20 bit registers.
- It is possible to enhance the memory size of code data or stack segments beyond 64 KB by allotting more than one segment for each area.

Memory Organization

- 8086, via its 20-bit address bus, can address 2^{20} = 1,048,576 or 1 MB of different memory locations.
- The memory map of 8086 is shown in Fig. 12.1, where the whole memory space starting from 00000 H to FFFFF H is **divided into 16 blocks—each one consisting of 64 KB.**
- This division is arbitrary but at the same time a convenient one—because the most significant hex digit increases by 1 with each additional block.
- The lower and upper ends of the memory map are shown separately—earmarking some spaces as reserved and some as 'dedicated'.
- The reserved locations are meant for future hardware and software needs while the dedicated locations are used for processing of specific system interrupts and reset functions.


FFFFFH
FFFF0H
F0000H
E0000H
D0000H
C0000H
B0000H
A0000H
90000H
80000H
70000H
60000H
50000H
40000H
30000H
20000H
10000H
0007FH
64K bytes
FFFFFH
Reserved
FFFFBH
Dedicated
FFFF0H
16 bytes
0007FH
Reserved
00013H
Dedicated
128 bytes

Q. Describe memory segmentation scheme of 8086. What is meant by currently active segments?

- 1 MB memory of 8086 is partitioned into 16 segments—each segment is of 64 KB length. Out of these 16 segments, only 4 segments can be active at any given instant of time— these are code segment, stack segment, data segment and extra segment
- The four memory segments that the CPU works with at any time are called currently active segments.
- Corresponding to these four segments, the registers used are Code Segment Register (CS), Data Segment Register (DS), Stack Segment Register (SS) and Extra Segment Register (ES) respectively.
- Each of these four registers is 16-bits wide and user accessible—i.e., their contents can be changed by software.

Memory Organization

- The code segment contains the instruction codes of a program, while data, variable and constants are held in data segment.
- **The stack segment is used to store interrupt and subroutine return addresses.**
- The extra segment contains the destination of data for certain **string instructions**. Thus 64 KB are available for program storage (in CS) as well as for stack (in SS) while 128 KB of space can be utilised for data storage (in DS and ES)

Q. Describe how the 20-bit physical address is generated?

- The 20-bit physical (real) address is generated by combining the offset (residing in IP, BP, SP, BX, SI or DI) and the content of one of the segment registers CS, DS, ES or SS.
- The process of combination is as follows:
- The content of the segment register is internally appended with 0 H (0000 H) on its right most end to form a 20-bit memory address—this 20-bit address points to the start of the segment.
- The offset is then added to the above to get the physical address.
- Fig. 12.3 shows pictorially the actual process of generating a 20-bit physical address.

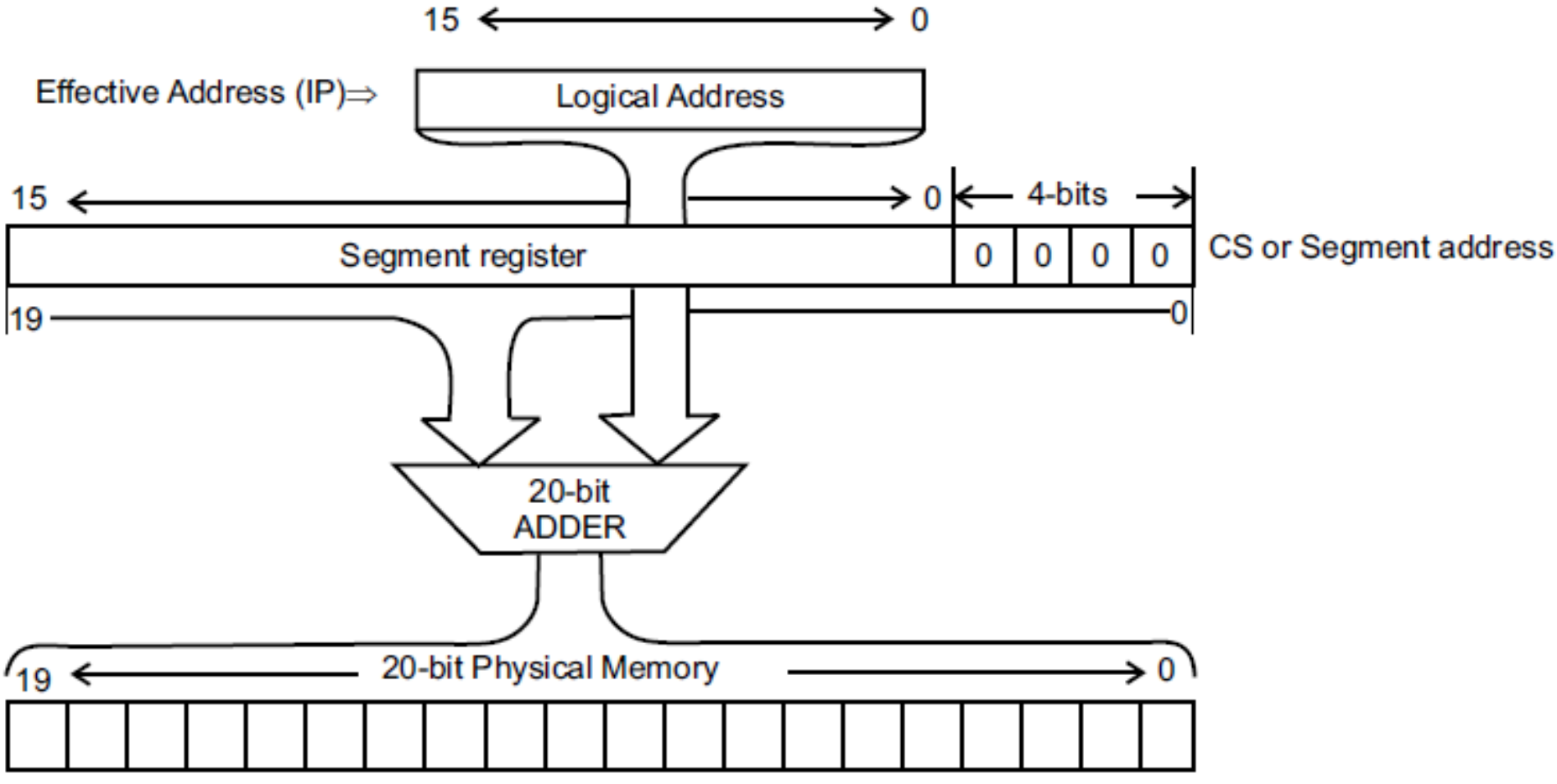


Fig. 12.3: Physical address generation

Thus, Physical Address = Segment Register content 16 D + Offset.

Segments, Segment Registers & Offset Registers

Segment Size = 64KB

Maximum number of segments possible = 14

Logical Address – 16 bits

Physical Address – 20 bits

2 Logical Addresses for each Segments.

- Base Address (16 bits)
- Offset Address (16 bits)

Segment registers are used to store the Base address of the segment.

Q. Discuss logical address, base segment address and physical address.

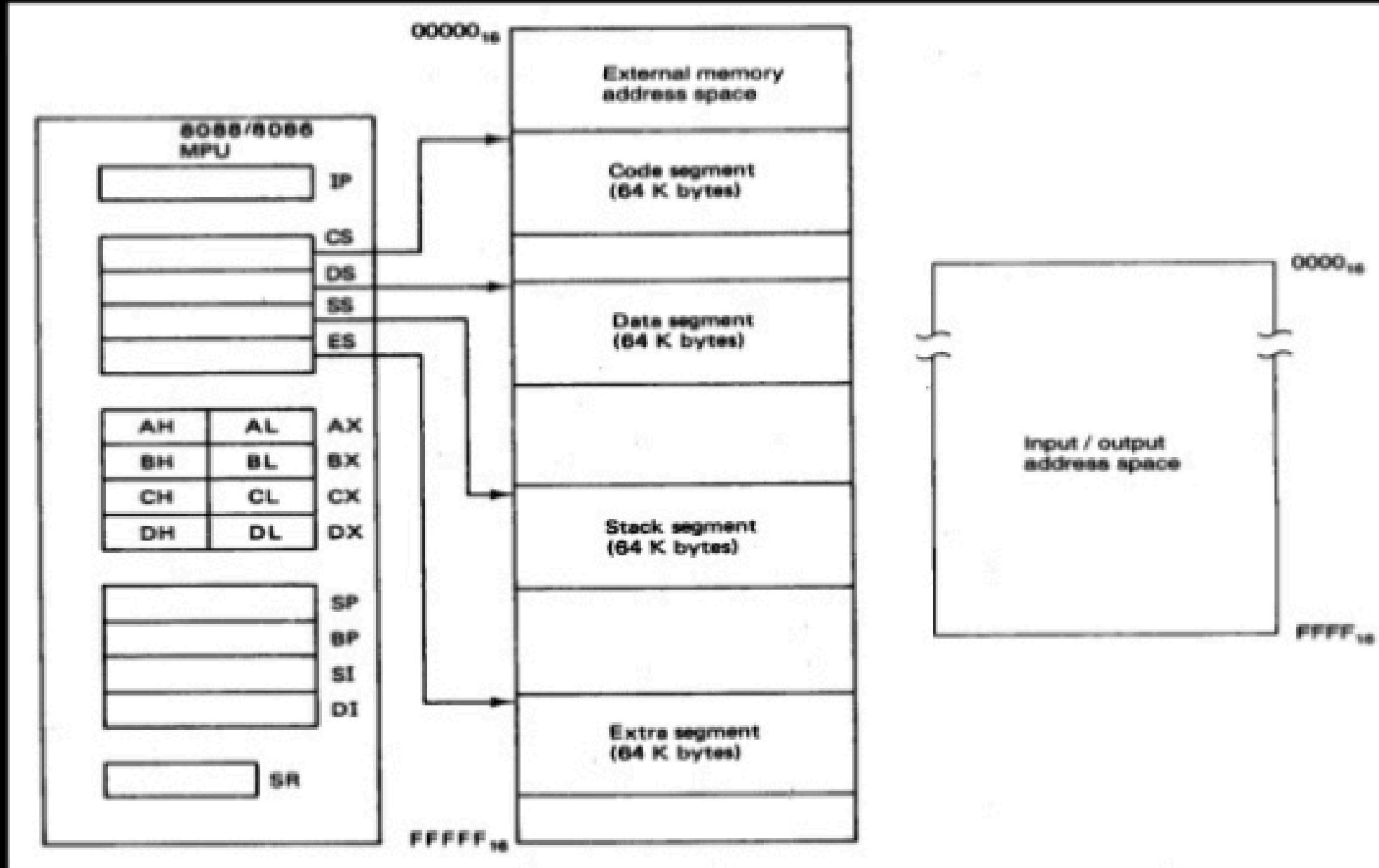
- The logical address, also goes by the name of effective address or offset address (also known as offset), is contained in the 16-bit IP, BP, SP, BX, SI or DI.
- The 16-bit content of one of the four segment registers (CS, DS, ES, SS) is known as the base segment address.
- Offset and base segment addresses are combined to form a 20-bit physical address (also called real address) that is used to access the memory.
- This 20-bit physical address is put on the address bus (AD19 – AD0) by the BIU.

Describe memory segmentation scheme of 8086. What is meant by currently active segments?

- 1 MB memory of 8086 is partitioned into 16 segments—each segment is of 64 KB length.
- Out of these 16 segments, only 4 segments can be active at any given instant of time— these are code segment, stack segment, data segment and extra segment.
- The four memory segments that the CPU works with at any time are called ***currently active segments***.
- Corresponding to these four segments, the registers used are Code Segment Register (CS), Data Segment Register (DS), Stack Segment Register (SS) and Extra Segment Register (ES) respectively.
- Each of these four registers is 16-bits wide and user accessible —i.e., their contents can be changed by software.

- The **code segment** contains the instruction codes of a program, while data, variables and constants are held in **data segment**.
- The **stack segment** is used to store interrupt and subroutine return addresses.
- The **extra segment** contains the destination of data for certain string instructions.
- Thus 64 KB are available for program storage (in CS) as well as for stack (in SS) while 128 KB of space can be utilised for data storage (in DS and ES).

Segmented Memory Representation



Mention the maximum size of memory that can be active for 8086.

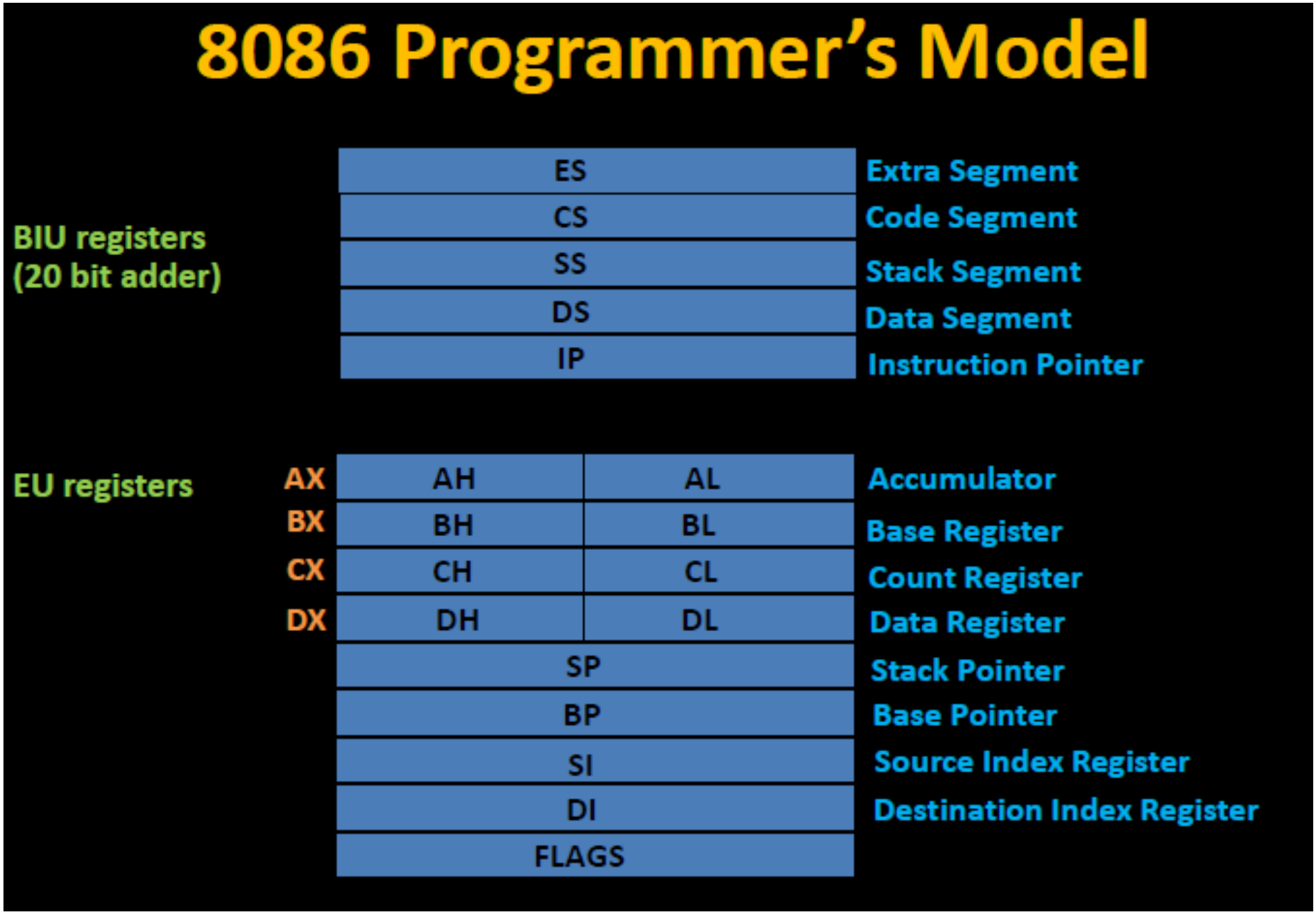
The maximum size of active memory for 8086 is 256 KB. The break-up being

- 64 KB for program
- 64 KB for stack and
- 128 MB for data.(DS+ES)

Programming model of the 8086

- The programming model of the 8086 is considered to be program visible because its registers are used during application programming and are specified by the instructions.
- Figure below illustrates the programming model of 8086 microprocessor.
- Some registers are general-purpose or multipurpose registers, while some have special purposes

8086 Programmer's Model



Programming Model

- Figure 2-1 illustrates the programming model of the 8086 through the Pentium II microprocessor.

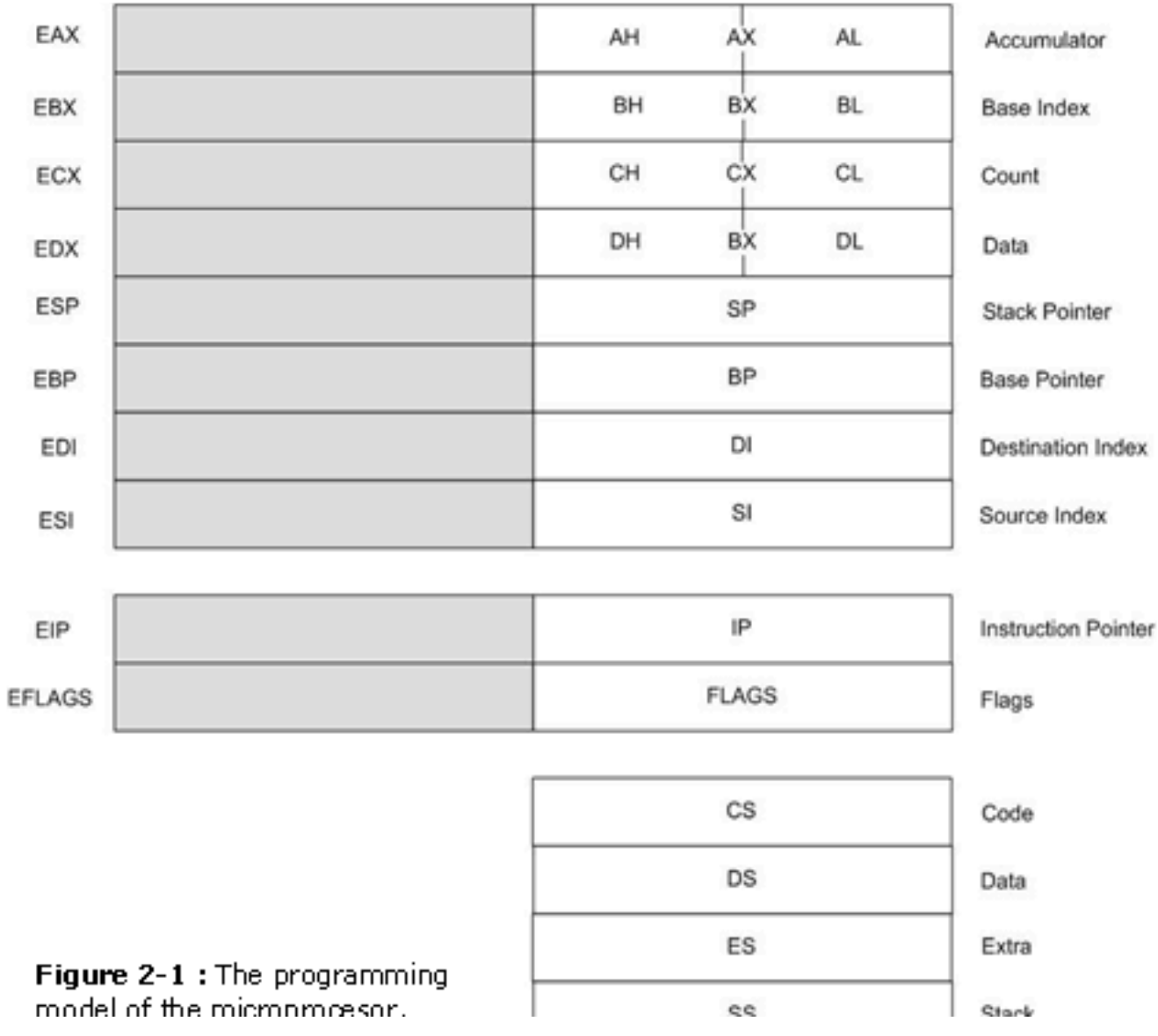


Figure 2-1 : The programming model of the microprocesor.

Programming Model

- The programming model contains 8-, 16-, and 32-bit registers. The 8-bit registers are **AH**, **AL**, **BH**, **BL**, **CH**, **CL**, **DH**, and **DL** and are referred to when an instruction is formed using these two-letter designations.
- The 16-bit registers are **AX**, **BX**, **CX**, **DX**, **SP**, **BP**, **DI**, **SI**, **IP**, **FLAGS**, **CS**, **DS**, **ES**, **SS** The **extended 32-bit registers** are **EAX**, **EBX**, **ECX**, **EDX**, **ESP**, **EBP**, **EDI**, **ESI**, **EIP**, and **EFLAGS**. These 32-bit extended registers, and 16-bit registers ES and GS are available only in the 80386 and above.
- Some registers are general-purpose or multipurpose registers, while some have special purposes. The **multipurpose** registers include EAX, EBX, ECX, EDX, EBP, EDI, and ESI.
- These registers hold various data sizes (bytes, words, or doublewords) and are used for almost any purpose, as dictated by a program.

Real Mode Memory Addressing

- The 80286 and above operate in either the real or protected mode.
- Only the 8086 and 8088 operate exclusively in the real mode.
- **Real mode operation** allows the microprocessor to address only the first 1M byte of memory space-even if it is the Pentium II microprocessor.
- Note that the first 1 M byte of memory is called either the **real memory** or **conventional memory** system.
- The DOS operating system requires the microprocessor to operate in the real mode.
- Real mode operation allows application software written for the 8086/8088, which contain only 1 M byte of memory, to function in the 80286 and above without changing the software.
- The upward compatibility of software is partially responsible for the continuing success of the Intel family of microprocessors.
- In all cases, each of these microprocessors begins operation in the real mode by default whenever power is applied or the microprocessor is reset.

Segments And Offsets

- A combination of a segment address and an offset address, access a memory location in the real mode.
- All real mode memory addresses must consist of a segment address plus an offset address.
- The **segment address,** located within one of the segment registers, defines the beginning address of any 64K-byte memory segment.
- The **offset address** selects any location within the 64K byte memory segment.
- Segments in the real mode always have a length of 64K bytes.
- Figure 2-3 shows how the segment plus offset addressing scheme selects a memory location.
- This illustration shows a memory segment that begins at location 1 0000H and ends at location 1 FFFEH 64K bytes in length.
- It also shows how an offset address, sometimes called a displacement, of F000H selects location 1F000H in the memory system.
- Note that the offset or displacement is the distance above the start of the segment, as shown in Figure 2-3.

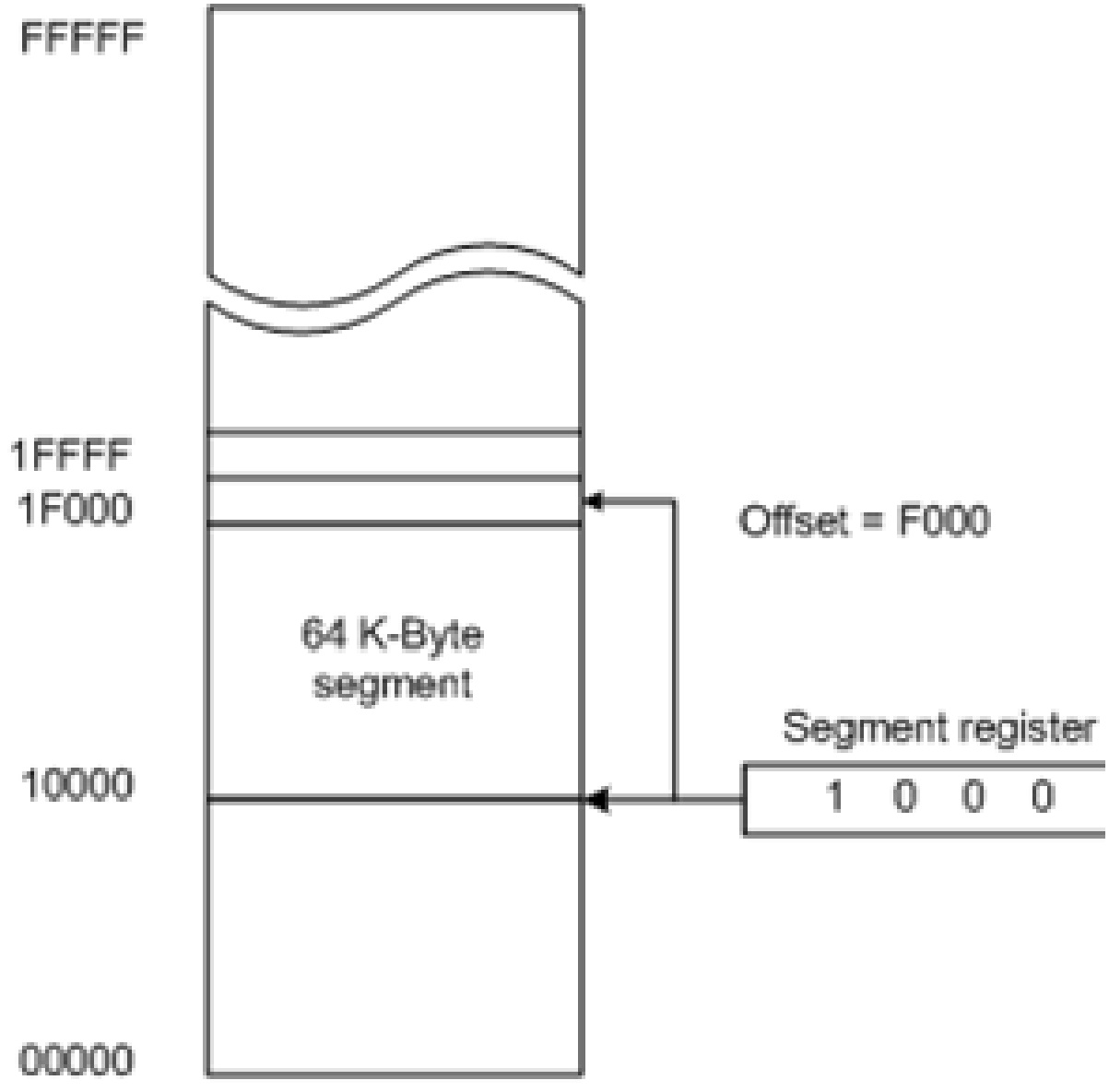


Figure 2-3 : The real mode memory-addressing scheme

Segments and Offsets

- The segment register in Figure 2-3 contains a 1000H, yet it addresses a starting segment at location 10000H.
- In the real mode, each segment register is internally appended with a 0H on its rightmost end. This forms a 20-bit memory address, allowing it to access the start of a segment.
- The microprocessor must generate a 20-bit memory address to access a location within the first 1 M of memory.
- For example, when a segment register contains a 1200H, it addresses a 64K-byte memory segment beginning at location 12000H.
- Likewise, if a segment register contains a 1201H, it addresses a memory segment beginning at location 12010H.

Segment	Offset	Special Purpose
CS	IP	Instruction address
SS	SP or BP	Stack address
DS	BX, DI, SI, an 8-bit number, or a 16-bit number	Data address
ES	DI for string instructions	String destination address

Table 2-1: Default 16-bit segment and offset address combinations

Introduction to Protected Mode Memory Addressing

- Protected mode memory addressing (80286 and above) allows access to data and programs located above the first 1M byte of memory, as well as within the first 1M byte of memory.
- Addressing this extended section of the memory system requires a change to the segment plus an offset addressing scheme used with real mode memory addressing.
- When data and programs are addressed in extended memory, the offset address is still used to access information located within the memory segment.
- One difference is that the segment address, is no longer present in the protected mode.
- In place of the segment address, the segment register contains a selector that selects a descriptor from a descriptor table.

Introduction to Protected Mode Memory Addressing

- The descriptor describes the memory segment's location, length, and access rights.
- Because the segment register and offset address still access memory, protected mode instructions are identical to real mode instructions.
- In fact, most programs written to function in the real mode will function without change in the protected mode.
- **The difference between modes is in the way that the segment register is interpreted by the microprocessor to access the memory segment.**
- Another difference, in the 80386 and above, is that the offset address can be a 32-bit number instead of a 16-bit number in the protected mode. A 32-bit offset address allows the microprocessor to access data within a segment that can be up to 4G bytes in length.

Selectors And Descriptors

- The selector, located in the segment register, selects one of 8192 descriptors from one of two tables of descriptors.
- The **descriptor** describes the location, length, and access rights of the segment of memory.
- Indirectly, the segment register still selects a memory segment, but not directly as in the real mode. For example, in the real mode, if CS = 0008H, the code segment begins at location 00080H.
- In the protected mode, this segment number can address any memory location in the entire system for the code segment.

Selectors And Descriptors

- There are two descriptor tables used with the segment registers: one contains global descriptors and the other contains local descriptors.
- The **global descriptors** contain segment definitions that apply to all programs, while the **local descriptors** are usually unique to an application.
- A global descriptor is a system descriptor and local descriptor is an **application descriptor.**
- Each descriptor table contains 8192 descriptors, so a total of 16,384 total descriptors are available to an application at any time.
- Because the descriptor describes a memory segment, this allows up to 16,384 memory segments to be described for each application.

- Figure 2-6 shows the format of a descriptor for the 80286 through the Pentium II.
- Note that each descriptor is 8 bytes in length, so the global and local descriptor tables are each a maximum of 64K bytes in length. Descriptors for the 80286 and the 80386 through the Pentium II differ slightly,

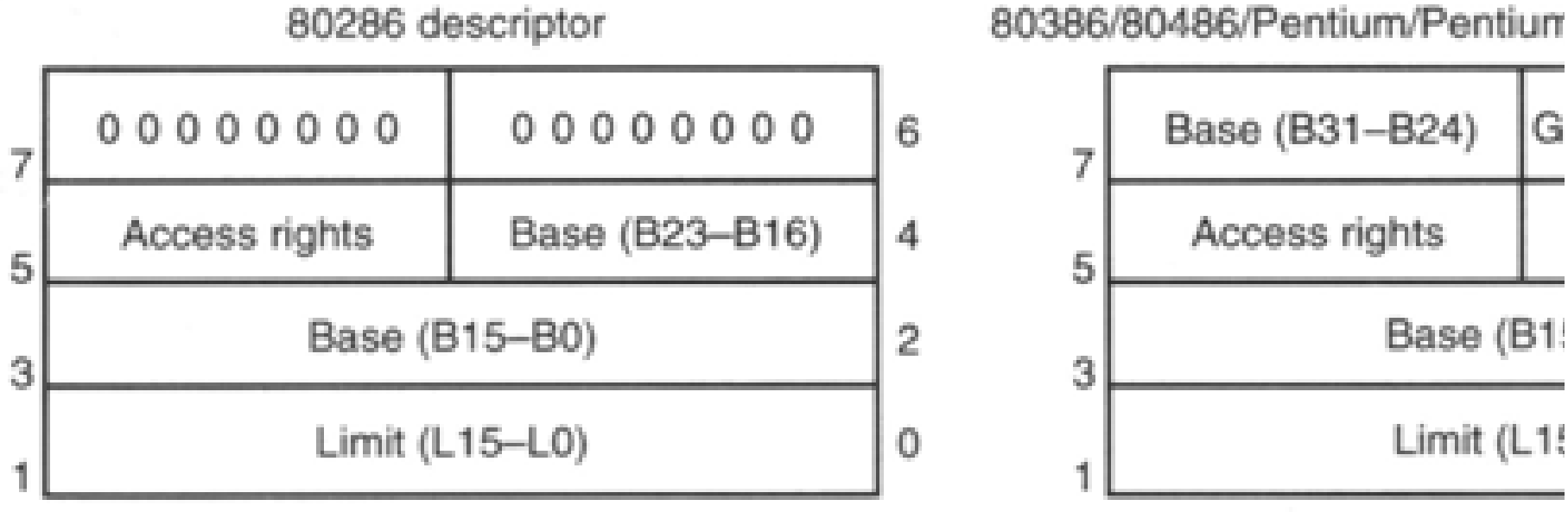


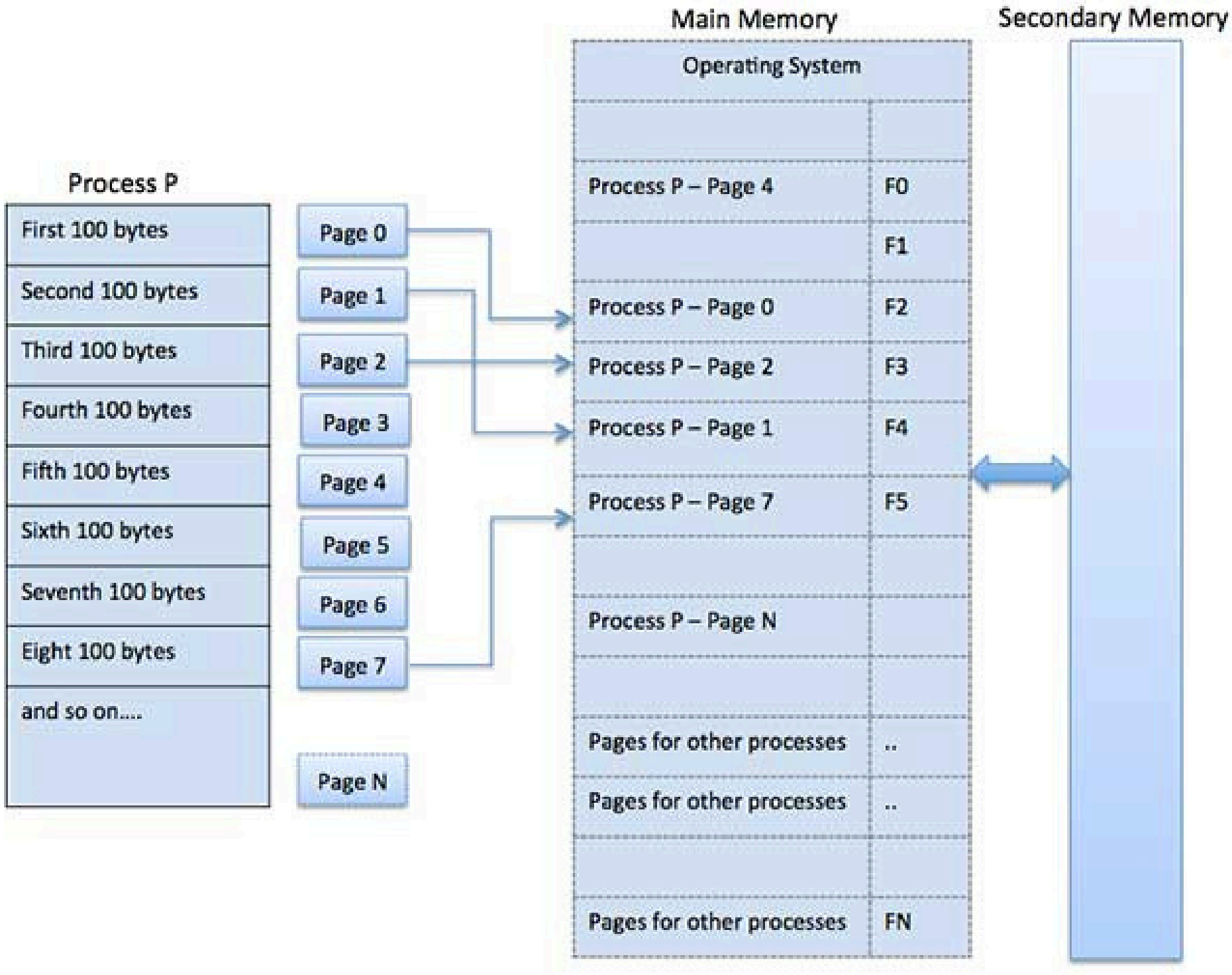
Figure 2-6 : The descriptor formats for the 80286 and 80386 / 80486 / P
Pentium II microprocessors.

Difference between Real and Protected Mode

Real Mode	Protected Mode (PVAM)
Memory addressing up to 1 MB physical memory	Memory addressing up to 16 MB of physical memory
No virtual memory support	Supports up tp to 64TB of virtual memory
Memory Protection mechanism is not available	Memory Protection Mechanism is avilable
Does not support virtual address space	Gives virtual and physical address space
Does not support LDT and GDT	Supports LDT and GDT
Segment descriptor cache is not available	Segment descriptor cache is available
Supports Segmentation	Supports segmentation and paging.

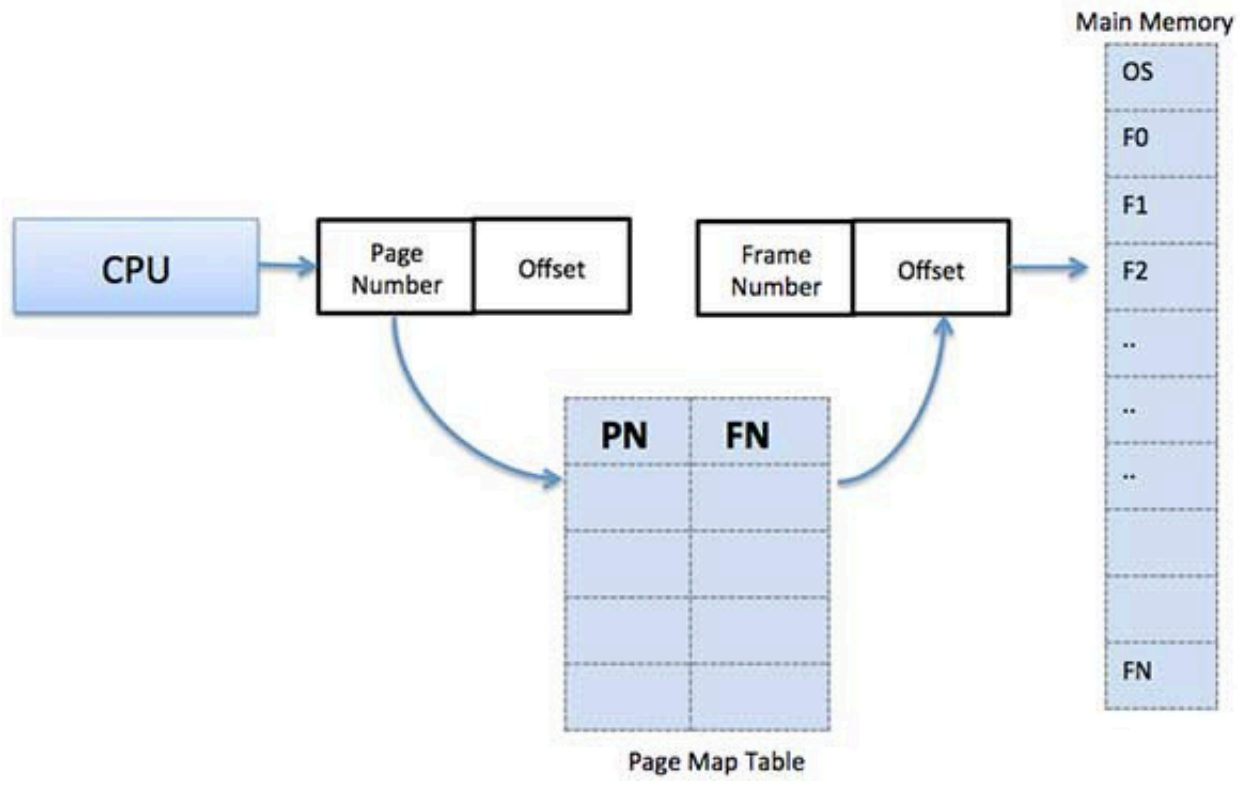
Paging

- A computer can address more memory than the amount physically installed on the system.
- This extra memory is actually called virtual memory and it is a section of a hard disk that's set up to emulate the computer's RAM.
- Paging technique plays an important role in implementing virtual memory.
- Paging is a memory management technique in which process address space is broken into blocks of the same size called **pages.**
- The size of the process is measured in the number of pages.
- Similarly, main memory is divided into small fixed-sized blocks of (physical) memory called **frames** and the size of a frame is kept the same as that of a page to have optimum utilization of the main memory.


Main Memory
Secondary Memory
Operating System
Process P
First 100 bytes
Second 100 bytes
Third 100 bytes
Fourth 100 bytes
Fifth 100 bytes
Sixth 100 bytes
Seventh 100 bytes
Eight 100 bytes
and so on....
Page 0
Page 1
Page 2
Page 3
Page 4
Page 5
Page 6
Page 7
Page N
Process P – Page 4
F0
F1
Process P – Page 0
F2
Process P – Page 2
F3
Process P – Page 1
F4
Process P – Page 7
F5
Process P – Page N
Pages for other processes
..
Pages for other processes
..
Pages for other processes
FN

Address Translation

- Page address is called **logical address** and represented by **page number** and the **offset**.
- Logical Address = Page number + page offset Frame address is called **physical address** and represented by a **frame number** and the **offset**.
- Physical Address = Frame number + offset A data structure called **page map table** is used to keep track of the relation between a page of a process to a frame in physical memory.



Paging…

- When the system allocates a frame to any page, it translates this logical address into a physical address and create entry into the page table to be used throughout execution of the program.
- When a process is to be executed, its corresponding pages are loaded into any available memory frames.
- Suppose you have a program of 8Kb but your memory can accommodate only 5Kb at a given point in time, then the paging concept will come into picture.
- When a computer runs out of RAM, the operating system (OS) will move idle or unwanted pages of memory to secondary memory to free up RAM for other processes and brings them back when needed by the program.
- This process continues during the whole execution of the program where the OS keeps removing idle pages from the main memory and write them onto the secondary memory and bring them back when required by the program.

Advantages and Disadvantages of Paging

- Here is a list of advantages and disadvantages of paging −

i. Paging reduces external fragmentation.
ii. Paging is simple to implement and assumed as an efficient memory management technique.
iii. Due to equal size of the pages and frames, swapping becomes very easy.
iv. Page table requires extra memory space, so may not be good for a system having small RAM.

Addressing mode

- An **addressing mode** refers to how you are addressing a given memory location
- These are
  1. Data addressing modes,
  2. Memory addressing modes, and
  3. Stack addressing.

Data-addressing modes

- The **data-addressing modes** include
  1. register,
  2. immediate,
  3. direct,
  4. register indirect

Program memory-addressing modes

- The program **memory-addressing modes** include
  1. program relative,
  2. direct, and
  3. indirect.
- The operation of the **stack memory** is explained so that the PUSH and POP instructions are understood.

3. Data-Addressing Modes

- Because the MOV instruction is a common and flexible instruction, it provides a basis for the explanation of the data-addressing modes.
- From Figure below, The **source** is to the right and the **destination** is to the left, next to the opcode MOV.
- An **opcode,** or operation code, tells the microprocessor which operation to perform.
- In Figure 3-1, the MOV AX,BX instruction transfers the word contents of the source register (BX) into the destination register (AX). The source never changes, but the destination usually changes.1 It is essential to remember that a MOV instruction always *copies* the source data and into the destination. The MOV

3.1 Register addressing

- Register addressing is the most common form of data addressing.
- The microprocessor contains the following 8-bit registers used with register addressing: AH, AL, BH, BL, CH, CL, DH, and DL. Also present are the following 16-bit registers: AX, BX, CX, DX, SP, BP, SI, and DI.
- In the 80386 and above, the extended 32-bit registers are EAX, EBX, ECX, EDX, ESP, EBP, EDI, and ESI.
- It is important for instructions to use registers that are the same size

3.1 Register addressing

- Table 3-1 shows some variations of register move instructions. A segment-to-segment register MOV instruction is not allowed.

ASSEMBLY	SIZE	OPERATION
MOV AL,BL	8-BITS	COPIES BL INTO AL
MOV CH,CL	8-BITS	COPIES CL INTO CH
MOV AX,CX	16-BITS	COPIES CX INTO AX
MOV SP,BP	16-BITS	COPIES BP INTO SP
MOV DS,AX	16-BITS	COPIES AX INTO DS
MOV SI,DI	16-BITS	COPIES DI INTO SI
MOV BX,ES	16-BITS	COPIES ES INTO BX
MOV ECX,EBX	32-BITS	COPIES ECX INTO EBX
MOV ES,DS	-	NOT ALLOWED (SEGMENT-TO SEGMENT)
MOV BL,DX	-	NOT ALLOWED (MIXED SIZES)
MOV CS,AX	-	NOT ALLOWED (CS SEGMENT REGISTER MAY NOT BE THE DESTINATION REGISTER)

3.2. Immediate Addressing

- It is termed as **immediate** because 8-bit data is transferred immediately to the accumulator (destination operand).
- The data is provided in the instruction
- Let's begin with an example.

  MOV A, #6A H

  In general, we can write,

  MOV A, #data

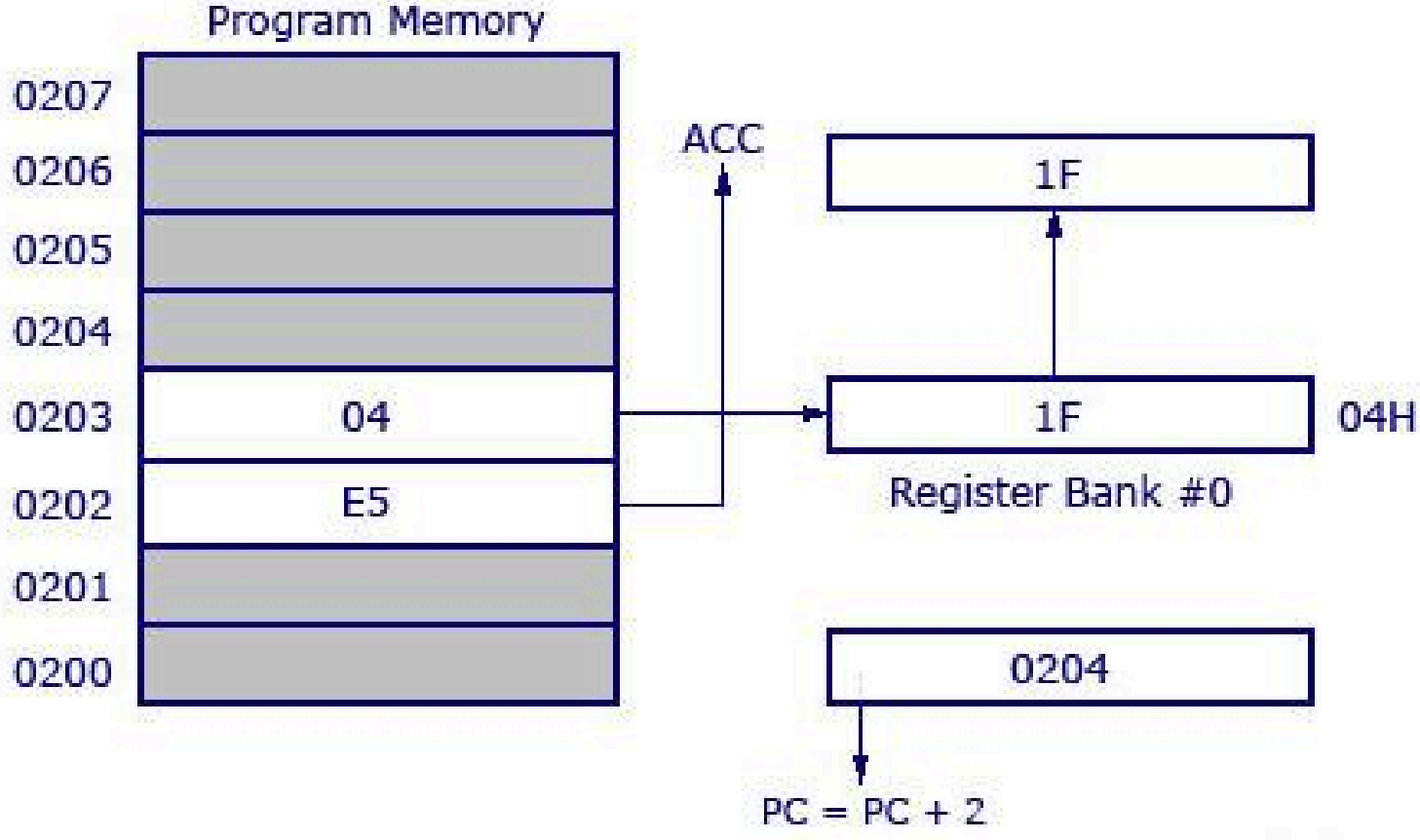
Direct Data Addressing

- This is another way of addressing an operand. Here, the address of the data (source data) is given as an operand.
- Let's take an example.

  MOV A, 04H
- The register bank#0 (4th register) has the address 04H.
- When the MOV instruction is executed, the data stored in register 04H is moved to the accumulator.
- As the register 04H holds the data 1FH, 1FH is moved to the accumulator.

Direct Addressing Mode

Instruction	Opcode	Bytes	Cycles
MOV A, #04H	E5	2	1



Register Indirect Addressing Mode

- In this addressing mode, the address of the data is stored in the register as operand.
- MOV A, @R0 Here the value inside R0 is considered as an address, which holds the data to be transferred to the accumulator.
- **Example**: If R0 has the value 20H, and data 2FH is stored at the address 20H, then the value 2FH will get transferred to the accumulator after executing this instruction.

STACK MEMORY-ADDRESSING MODES

- The stack plays an important role in all microprocessors.
- It holds data temporarily and stores return addresses for procedures.
- The stack memory is a LIFO (last-in, **first-out) memory, which** describes the way that data are stored and removed from the stack.
- Data are placed onto the stack with a **PUSH instruction and** removed with *a* **POP instruction.**
- The stack memory is maintained by two registers: the stack pointer (SP) and the stack segment register (SS).
- The SP register always points to an area of memory located within the stack segment.
- The SP register adds to SS + OH to form the stack memory address in the real mode. In protected mode operation, the SS register holds a selector that accesses a descriptor for the base address of the stack segment.

Thank You